Przewodnik wdrożeniowy TestGEN

Spis treści

- Dokumentacja wdrożeniowa — indeks
  - Dokumenty
  - Powiązane
- Getting Started — TestGEN (P1)
  - Wymagania
  - 1. Klonowanie
  - 2. Konfiguracja `.env`
  - 3. Uruchomienie środowiska
  - 4. Migracje bazy
  - 5. Weryfikacja
  - 6. Testy
  - Pierwszy przepływ (API)
  - Rozwiązywanie problemów
- Konfiguracja środowiska — TestGEN (P1)
  - Serwisy Docker Compose
  - OS-specific
  - Zmienne środowiskowe (`.env`)
  - Troubleshooting
  - Powiązane
- Podręcznik wdrożeniowy — instalacja i konfiguracja
  - Spis treści
  - Konwencje dokumentu
  - 1. Wymagania systemowe
  - 2. Instalacja single-node (Docker Compose)
  - 3. Instalacja multi-node (Kubernetes)
  - 4. Konfiguracja SSL/TLS
  - 5. Konfiguracja bazy danych (PostgreSQL)
  - 6. Konfiguracja Redis
  - 7. Konfiguracja RabbitMQ
  - 8. Konfiguracja MinIO / S3
  - Powiązane dokumenty
- Podręcznik operacyjny — backup, skalowanie, aktualizacje, monitoring, DR
  - Spis treści
  - Format procedur
  - 1. Kopie zapasowe i przywracanie

2. Skalowanie horyzontalne
3. Aktualizacje systemu
4. Monitoring
5. Odtwarzanie po awarii (DR)
Powiązane dokumenty

Dokumentacja wdrożeniowa — indeks

> Uruchamianie i utrzymanie środowisk TestGEN.

Dokumenty

Dokument	Zakres
getting-started.md	Pierwsze uruchomienie lokalne (dev): monorepo, zależności, docker compose
environment-setup.md	Konfiguracja środowiska: zmienne env, sekrety, serwisy
01-instalacja.md	**Instalacja produkcyjna:** wymagania systemowe, Docker Compose (single-node), Kubernetes/Helm (multi-node), SSL/TLS, konfiguracja PostgreSQL, Redis, RabbitMQ, MinIO
02-operacje.md	**Operacje produkcyjne:** backup/restore (PostgreSQL, Redis, MinIO), skalowanie horyzontalne, aktualizacje (rolling/blue-green/canary), monitoring (Prometheus + Grafana + Loki), disaster recovery
examples/	Pliki konfiguracyjne i skrypty: `.env.production`, `postgresql.conf`, `redis.conf`, `rabbitmq.conf`, `pgbouncer.ini`, `values.yaml`, `backup-*.sh`, `restore-postgres.sh`, `prometheus-rules.yml`, `grafana-dashboard.json`

Powiązane

Runbook operacyjny webhooków płatności: ../troubleshooting/tpay-sandbox-runbook.md oraz ../architecture/billing/04-webhooki.md.

Getting Started — TestGEN (P1)

Przewodnik dla nowego dewelopera: od sklonowania repo do działającego środowiska. Cel: **< 30 minut**. Szczegóły środowiska: environment-setup.md; architektura: architecture.md.

Wymagania

Narzędzie	Wersja
Docker Desktop (z Compose v2)	aktualna
Git	aktualna
Python (backend lokalnie)	3.12+
Node.js (frontend lokalnie)	22+
(opcjonalnie) Flutter	3.38 (klient P3)

Pełne środowisko deweloperskie uruchamia **Docker Compose** — Python/Node potrzebne tylko do pracy poza kontenerami (lint, testy natywne).

1. Klonowanie

```
git clone <repo-url> testgen
cd testgen
```

2. Konfiguracja `.env`

```
cp .env.example .env
```

Wygeneruj parę kluczy RS256 (wymagana do JWT — ADR-003):

```
mkdir -p backend/keys
openssl genrsa -out backend/keys/private.pem 2048
openssl rsa -in backend/keys/private.pem -pubout -out
        backend/keys/public.pem
```

Klucz MFA (Fernet):

```
python -c "from cryptography.fernet import Fernet;
        print(Fernet.generate_key().decode())"
# wklej wynik do .env jako ENCRYPTION_KEY=
```

Opis wszystkich zmiennych: environment-setup.md.

3. Uruchomienie środowiska

```
make dev      # docker compose -f docker-compose.yml -f docker-
        compose.dev.yml up --build
```

Uruchamia 8 serwisów: `backend`, `frontend`, `celery-worker`, `flower`, `postgres`, `redis`, `rabbitmq`, `minio`. Skróty: `make logs`, `make down`, `make clean`.

4. Migracje bazy

```
make migrate   # alembic upgrade head (wewnątrz backend)
```

5. Weryfikacja

```
curl http://localhost:8000/api/v1/health    # → 200
# Frontend:      http://localhost:4200
# Swagger UI:    http://localhost:8000/docs
# ReDoc:         http://localhost:8000/redoc
# MinIO console: http://localhost:9001
# RabbitMQ mgmt: http://localhost:15672
```

6. Testy

```
make test          # backend: pytest
cd frontend && npx ng test     # frontend: Karma/Vitest
```

CI lustrują: `make ci-lint`, `make ci-test`, `make ci-build`.

Pierwszy przepływ (API)

```
# 1. Rejestracja + logowanie (cookie access_token)
curl -X POST http://localhost:8000/api/v1/auth/register \
  -H "Content-Type: application/json" \
  -d
        '{"email":"dev@example.com","password":"SecurePass123!","full_na
        me":"Dev"}'

curl -i -X POST http://localhost:8000/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"dev@example.com","password":"SecurePass123!"}'
# zapisz ciasteczko access_token z nagłówka Set-Cookie

# 2. Projekt + upload dokumentacji (async)
curl -X POST http://localhost:8000/api/v1/projects \
  -H "Cookie: access_token=…" -H "Content-Type: application/json" \
  -d '{"name":"Demo"}'

curl -X POST http://localhost:8000/api/v1/projects/{PID}/upload-documents
        \
  -H "Cookie: access_token=…" -F "file=@specyfikacja.pdf" -F
        "file_category=documentation"
# → 202, odpytuj GET /documents/{DID}/status aż ready
```

Więcej przykładów: api/examples/.

Rozwiązywanie problemów

Objaw	Rozwiązanie
`make dev` — port zajęty	`make down` i zwolnij port (5432/6379/5672/9000/4200/8000)
Health `502` po starcie	odczekaj `start_period` (backend ~15s); `make logs backend`
Błędy RLS w testach	test musi ustawić kontekst `app.current_tenant`
Migracja się cofa	`make migrate`; sprawdź `backend/alembic/versions/`

Szczegóły: environment-setup.md.

Konfiguracja środowiska — TestGEN (P1)

Szczegółowy opis serwisów, portów i zmiennych środowiskowych. Szybki start: getting-started.md.

Serwisy Docker Compose

`docker-compose.yml` (+ `docker-compose.dev.yml` dla dev) — 8 serwisów:

Serwis	Obraz	Porty	Rola
`backend`	budowany (`./backend`, target production)	8000	FastAPI API
`frontend`	budowany (`./frontend`)	4200→80	Angular SPA
`celery-worker`	= backend	—	worker Celery (`-Q default,analysis,export,security`)
`flower`	mher/flower:2.0	5555	monitoring tasków Celery (basic-auth `FLOWER_USER` / `FLOWER_PASSWORD`)
`postgres`	postgres:16-alpine	5432	baza + RLS
`redis`	redis:7-alpine	6379	result backend Celery + cache
`rabbitmq`	rabbitmq:3.12-management-alpine	5672, 15672	broker zadań
`minio`	minio/minio	9000, 9001	storage S3 binariów

Wolumeny: `postgres-data`, `redis-data`, `rabbitmq-data`, `minio-data`.

OS-specific

- **Windows**: Docker Desktop z WSL2; skrypty używają POSIX — używaj Git Bash lub WSL. Ścieżki kluczy w `.env` z ukośnikami w przód (`keys/private.pem`).
- **macOS**: Docker Desktop; colima jako alternatywa.
- **Linux**: Docker Engine + Compose v2 (natively).

Zmienne środowiskowe (`.env`)

Wszystkie z domyślnymi wartościami w `.env.example`. Skopiuj: `cp .env.example .env`.

Aplikacja / backend

Zmienna	Opis
`APP_NAME`, `APP_ENV`, `DEBUG`	nazwa, środowisko, tryb debug
`BACKEND_HOST`, `BACKEND_PORT`	bind serwera (0.0.0.0:8000)
`SECRET_KEY`	sekret aplikacji (zmień w produkcji)

JWT (RS256)

Zmienna	Opis
`JWT_PRIVATE_KEY_PATH`, `JWT_PUBLIC_KEY_PATH`	ścieżki kluczy PEM (ADR-003)
`JWT_ACCESS_TOKEN_EXPIRE_MINUTES`	TTL access tokenu (15)
`JWT_REFRESH_TOKEN_EXPIRE_DAYS`	TTL refresh tokenu (7)

Baza danych (PostgreSQL)

Zmienna	Opis
`POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_DB`, `POSTGRES_HOST`, `POSTGRES_PORT`	parametry PG
`DATABASE_URL`	`postgresql+asyncpg://…`

Redis / RabbitMQ / MinIO

Zmienna	Opis
`REDIS_HOST`, `REDIS_PORT`, `REDIS_URL`	cache + result backend
`RABBITMQ_USER`, `RABBITMQ_PASSWORD`, `RABBITMQ_HOST`, `RABBITMQ_PORT`, `RABBITMQ_URL`	broker
`MINIO_ROOT_USER`, `MINIO_ROOT_PASSWORD`, `MINIO_ENDPOINT`, `MINIO_USE_SSL`	storage

Celery

Zmienna	Opis
`CELERY_BROKER_URL`, `CELERY_RESULT_BACKEND`	broker (amqp) + backend (redis)
`CELERY_SOFT_TIME_LIMIT`, `CELERY_HARD_TIME_LIMIT`	limity czasu (300/330)
`CELERY_MAX_RETRIES`, `CELERY_RETRY_DELAYS`	ponowienia (3; 10/60/300 s)

Auth: MFA, ciasteczka, brute-force

Zmienna	Opis
`MFA_TOKEN_EXPIRE_MINUTES`, `ENCRYPTION_KEY`	TOTP + Fernet dla sekretów MFA
`COOKIE_SECURE`, `COOKIE_DOMAIN`	flagi HttpOnly cookie
`MAX_LOGIN_ATTEMPTS`, `ACCOUNT_LOCKOUT_MINUTES`, `LOGIN_ATTEMPTS_WINDOW_MINUTES`	blokada po nieudanych próbach

SMTP / email / CORS / admin

Zmienna	Opis
`SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASSWORD`, `SMTP_FROM_EMAIL`, `SMTP_USE_TLS`	wysyłka email
`EMAIL_VERIFICATION_URL_TEMPLATE`, `EMAIL_INVITATION_URL_TEMPLATE`	szablony linków
`CORS_ORIGINS`	dozwolone originy (dev: `localhost:4200,localhost:8000`)
`BOOTSTRAP_ADMIN`, `ADMIN_EMAIL`, `ADMIN_PASSWORD`	bootstrap platform admina (`__system__`)
`API_URL`	bazowy URL API dla frontendu

Troubleshooting

Problem	Rozwiązanie
Backend nie wstaje	`make logs backend`; sprawdź `DATABASE_URL`, obecność kluczy RSA
`celery-worker` nie pobiera zadań	sprawdź `RABBITMQ_URL`; `make celery-inspect`
MFA setup rzuca błąd	ustaw `ENCRYPTION_KEY` (Fernet)
RLS blokuje wszystko	kontekst `app.current_tenant` ustawiany po `/auth/select-tenant`
MinIO `connection refused`	odczekaj healthcheck; sprawdź `MINIO_ENDPOINT`
Porty zajęte	`make down`; zwolnij 5432/6379/5672/9000/4200/8000
Migracje się nie aplikują	`make migrate`; sprawdź `backend/alembic/versions/`

Powiązane

- getting-started.md, architecture.md.
- ADR-002, ADR-003, ADR-004.

Podręcznik wdrożeniowy — instalacja i konfiguracja

> **Wersja:** 1.0
> **Data:** 2026-09-16
> **Docelowy czytelnik:** inżynier DevOps / administrator systemów
> **Szacowany czas wdrożenia:** < 4 h (single-node) · < 8 h (multi-node Kubernetes)

Spis treści

1. Wymagania systemowe
    1. Wymagania sprzętowe
    2. Wymagania programowe
    3. Wymagania sieciowe
2. Instalacja single-node (Docker Compose)
    1. Przygotowanie systemu operacyjnego
    2. Pobranie i konfiguracja projektu
    3. Generowanie kluczy kryptograficznych
    4. Plik .env — konfiguracja zmiennych środowiskowych
    5. Uruchomienie systemu
    6. Inicjalizacja bazy danych i konto administratora
    7. Weryfikacja instalacji
    8. Rozwiązywanie problemów
3. Instalacja multi-node (Kubernetes)
    1. Wymagania klastra
    2. Struktura Helm chart
    3. Przestrzeń nazw i RBAC
    4. Zarządzanie sekretami
    5. Wolumeny trwałe (PV / PVC)
    6. Obiekt Ingress i routing
    7. Instalacja za pomocą Helm
    8. Automatyczne skalowanie (HPA)
    9. Weryfikacja wdrożenia
4. Konfiguracja SSL/TLS
    1. Let's Encrypt z cert-manager
    2. Własny urząd certyfikacji (CA)
    3. Terminacja HTTPS
5. Konfiguracja bazy danych (PostgreSQL)
    1. Instalacja i inicjalizacja
    2. Dostrajanie postgresql.conf
    3. Row-Level Security (RLS)
    4. Rozszerzenia: pgcrypto i uuid-ossp

    5. PgBouncer — pula połączeń
    6. Migracje (Alembic)
6. Konfiguracja Redis
    1. Tryb pracy i persistence
    2. Polityka eksmisji
    3. Weryfikacja
7. Konfiguracja RabbitMQ
    1. Kolejki i exchange
    2. Wirtualne hosty
    3. Wysoka dostępność (HA)
    4. Weryfikacja
8. Konfiguracja MinIO / S3
    1. Tworzenie zasobników (buckets)
    2. Polityki dostępu
    3. Reguły cyklu życia
    4. Weryfikacja

Konwencje dokumentu

Element	Znaczenie
`polecenie`	komenda do wykonania w terminalu
pogrubienie	wartość do dostosowania per środowisko
`# komentarz`	objaśnienie wewnątrz pliku konfiguracyjnego
⚠️	krok krytyczny — pominięcie skutkuje awarią

Pliki konfiguracyjne przykładowe znajdują się w katalogu `examples/`.

---

1. Wymagania systemowe

1.1. Wymagania sprzętowe

Single-node (Docker Compose) — minimalne

Komponent	vCPU	RAM	Dysk	Uwagi
FastAPI (backend)	2	4 GB	20 GB	serwer API + Uvicorn
PostgreSQL 16	4	8 GB	100 GB SSD	baza relacyjna + RLS
Redis 7	2	4 GB	—	pamięć podręczna + wyniki Celery
RabbitMQ 3.12+	2	4 GB	—	broker zadań
MinIO	2	4 GB	zmienny	magazyn plików (kod, dokumentacja, zrzuty)
Celery worker (×1)	4	8 GB	—	przetwarzanie analiz

Komponent	vCPU	RAM	Dysk	Uwagi
Łącznie (single-node)	**8**	**32 GB**	**200 GB SSD**	uwzgl. współdzielenie zasobów

> **Uwaga:** wartości minimalne zakładają ≤ 5 równoczesnych analiz. Dla obciążenia produkcyjnego (> 10 równoczesnych analiz) zalecane: 16 vCPU, 64 GB RAM, NVMe SSD.

Multi-node (Kubernetes) — minimalne per pod

Pod	Repliki	vCPU (request/limit)	RAM (request/limit)	Dysk (PVC)
backend	2	1 / 2	2 Gi / 4 Gi	—
frontend (nginx)	2	0.25 / 0.5	256 Mi / 512 Mi	—
celery-worker	2–6	2 / 4	4 Gi / 8 Gi	—
celery-beat	1	0.25 / 0.5	256 Mi / 512 Mi	—
flower	1	0.25 / 0.5	256 Mi / 512 Mi	—
PostgreSQL	1 (lub HA)	2 / 4	4 Gi / 8 Gi	100 Gi SSD
Redis	1	1 / 2	2 Gi / 4 Gi	10 Gi
RabbitMQ	1 (lub klaster)	1 / 2	2 Gi / 4 Gi	20 Gi
MinIO	1 (lub distributed)	1 / 2	2 Gi / 4 Gi	500 Gi+

1.2. Wymagania programowe

Oprogramowanie	Wersja minimalna	Uwagi
System operacyjny	AlmaLinux 9 / Ubuntu 22.04 LTS	64-bit, jądro ≥ 5.15
Docker Engine	24.0+	z wtyczką Compose v2
Docker Compose	2.20+	wbudowany w Docker Desktop
Kubernetes	1.28+	tylko multi-node
Helm	3.12+	tylko multi-node
kubectl	zgodny z klastrem	tylko multi-node
Git	2.39+	klonowanie repozytorium
OpenSSL	3.0+	generowanie kluczy RSA
curl	dowolna	weryfikacja

1.3. Wymagania sieciowe

Porty

Port	Protokół	Usługa	Dostępność
80	TCP	Frontend (nginx)	publiczny
443	TCP	HTTPS (reverse proxy)	publiczny
8000	TCP	Backend API (FastAPI)	wewnętrzny*
5432	TCP	PostgreSQL	wewnętrzny
6379	TCP	Redis	wewnętrzny
5672	TCP	RabbitMQ (AMQP)	wewnętrzny
15672	TCP	RabbitMQ Management	wewnętrzny
9000	TCP	MinIO API	wewnętrzny
9001	TCP	MinIO Console	wewnętrzny
5555	TCP	Flower (monitoring Celery)	wewnętrzny

> * W trybie single-node port 8000 jest przekierowywany przez nginx; w K8s — przez Ingress. Nie powinien być eksponowany publicznie.

DNS

- Domena publiczna (np. `testgen.example.com`) wskazująca na adres IP serwera / load balancera.
- Opcjonalne subdomeny: `api.testgen.example.com`, `minio.testgen.example.com`.

Zapora sieciowa

- Dozwolony ruch przychodzący: porty 80 i 443.
- Dozwolony ruch wychodzący: HTTPS (443) do API LLM (OpenAI, Anthropic), SMTP, tpay, Let's Encrypt.
- Komunikacja wewnętrzna: porty 5432, 6379, 5672, 9000 — wyłącznie w sieci prywatnej.

---

2. Instalacja single-node (Docker Compose)

2.1. Przygotowanie systemu operacyjnego

AlmaLinux 9:

```
# Aktualizacja pakietów
sudo dnf update -y

# Instalacja Docker Engine
sudo dnf install -y dnf-plugins-core
sudo dnf config-manager --add-repo
         https://download.docker.com/linux/rhel/docker-ce.repo
sudo dnf install -y docker-ce docker-ce-cli containerd.io docker-compose-
         plugin

# Uruchomienie i autostart
sudo systemctl enable --now docker

# Dodanie użytkownika do grupy docker
sudo usermod -aG docker $USER
newgrp docker
```

Ubuntu 22.04 LTS:

```
# Aktualizacja pakietów
sudo apt update && sudo apt upgrade -y

# Zależności
sudo apt install -y ca-certificates curl gnupg

# Klucz GPG i repozytorium Docker
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-
         by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Instalacja
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose-
         plugin

# Użytkownik w grupie docker
sudo usermod -aG docker $USER
newgrp docker
```

Weryfikacja:

```
docker --version        # Docker version 24.x+
docker compose version  # Docker Compose version v2.20+
```

2.2. Pobranie i konfiguracja projektu

```
# Klonowanie repozytorium
git clone https://bitbucket.org/nexio/testgen.git
cd testgen

# Skopiowanie szablonu zmiennych środowiskowych
cp .env.example .env
```

2.3. Generowanie kluczy kryptograficznych

⚠️ **Klucze RSA są wymagane do działania uwierzytelniania JWT (RS256).**

```
# Utworzenie katalogu na klucze
mkdir -p backend/keys

# Klucz prywatny RSA 4096-bit
openssl genrsa -out backend/keys/private.pem 4096

# Klucz publiczny
openssl rsa -in backend/keys/private.pem -pubout -out
         backend/keys/public.pem

# Ograniczenie uprawnień
chmod 600 backend/keys/private.pem
chmod 644 backend/keys/public.pem
```

⚠️ **Klucz szyfrowania Fernet (wymagany dla MFA):**

```
python3 -c "from cryptography.fernet import Fernet;
         print(Fernet.generate_key().decode())"
# Wynik skopiuj do ENCRYPTION_KEY w pliku .env
```

2.4. Plik `.env` — konfiguracja zmiennych środowiskowych

Edytuj plik `.env` i dostosuj poniższe wartości do środowiska produkcyjnego.

⚠️ **Zmienne wymagające zmiany w produkcji:**

```
# === Tryb aplikacji ===
APP_ENV=production
DEBUG=false

# === Sekret aplikacji (losowy ciąg ≥ 64 znaki) ===
SECRET_KEY=<wygeneruj: openssl rand -hex 64>

# === Baza danych — silne hasło ===
POSTGRES_PASSWORD=<silne_hasło_min_32_znaki>
DATABASE_URL=postgresql+asyncpg://testgen:<hasło>@postgres:5432/testgen

# === Redis ===
# W produkcji zalecane zabezpieczenie hasłem:
# REDIS_URL=redis://:haslo@redis:6379/0

# === RabbitMQ — silne hasło ===
RABBITMQ_PASSWORD=<silne_hasło>
RABBITMQ_URL=amqp://testgen:<hasło>@rabbitmq:5672//

# === MinIO — silne hasło ===
MINIO_ROOT_PASSWORD=<silne_hasło_min_16_znaków>

# === Celery — adresy spójne z powyższymi ===
CELERY_BROKER_URL=amqp://testgen:<hasło_rmq>@rabbitmq:5672//
CELERY_RESULT_BACKEND=redis://redis:6379/1

# === JWT — ścieżki kluczy ===
JWT_PRIVATE_KEY_PATH=keys/private.pem
JWT_PUBLIC_KEY_PATH=keys/public.pem

# === SMTP — dane serwera pocztowego ===
SMTP_HOST=smtp.example.com
SMTP_PORT=587
SMTP_USER=noreply@example.com
SMTP_PASSWORD=<hasło_smtp>
SMTP_FROM_EMAIL=noreply@example.com

# === Szyfrowanie MFA ===
ENCRYPTION_KEY=<klucz_fernet_z_kroku_2.3>

# === Ciasteczka — produkcja wymaga HTTPS ===
COOKIE_SECURE=true
COOKIE_DOMAIN=testgen.example.com

# === CORS — domena produkcyjna ===
CORS_ORIGINS=https://testgen.example.com

# === URL frontendu ===
FRONTEND_BASE_URL=https://testgen.example.com
API_URL=https://testgen.example.com/api/v1

# === Flower — zmień dane logowania ===
FLOWER_USER=<użytkownik>
FLOWER_PASSWORD=<silne_hasło>
```

```
# === Konto administratora platformy ===
BOOTSTRAP_ADMIN=true
ADMIN_EMAIL=admin@example.com
ADMIN_PASSWORD=<silne_hasło_min_12_znaków>
```

Pełna lista zmiennych: `environment-setup.md`. Plik wzorcowy: `examples/.env.production.example`.

2.5. Uruchomienie systemu

```
# Zbudowanie obrazów i uruchomienie wszystkich serwisów
docker compose up -d --build

# Śledzenie logów w czasie rzeczywistym
docker compose logs -f
```

Proces budowania trwa 5–15 minut (zależnie od łącza i wydajności maszyny).

2.6. Inicjalizacja bazy danych i konto administratora

Migracje bazy danych uruchamiają się **automatycznie** przy starcie kontenera `backend` (skrypt `docker-entrypoint.sh` wykonuje `alembic upgrade head`).

⚠️ Mechanizm blokady advisory (`pg_advisory_lock`) zapobiega wyścigowi migracji między kontenerami `backend` i `celery-worker`.

Jeśli ustawiono `BOOTSTRAP_ADMIN=true`, konto administratora platformy zostanie utworzone automatycznie przy pierwszym uruchomieniu (tenant `__system__`).

Ręczna weryfikacja migracji:

```
docker compose exec backend alembic current
# Powinno wyświetlić aktualną rewizję, np.: abc123def456 (head)

docker compose exec backend alembic history --verbose | head -20
```

2.7. Weryfikacja instalacji

```
# 1. Sprawdzenie statusu kontenerów
docker compose ps
# Wszystkie serwisy powinny mieć status "healthy" lub "running"

# 2. Healthcheck backendu
curl -s http://localhost:8000/api/v1/health | python3 -m json.tool
# Oczekiwany wynik: {"status": "ok", ...}

# 3. Sprawdzenie frontendu
curl -s -o /dev/null -w "%{http_code}" http://localhost:4200
# Oczekiwany wynik: 200

# 4. Sprawdzenie RabbitMQ Management
curl -s -u testgen:<hasło> http://localhost:15672/api/overview | python3
        -m json.tool

# 5. Sprawdzenie MinIO
curl -s http://localhost:9000/minio/health/live
# Oczekiwany wynik: (pusta odpowiedź, kod 200)

# 6. Sprawdzenie Flower
curl -s -o /dev/null -w "%{http_code}" http://localhost:5555
# Oczekiwany wynik: 200 (po uwierzytelnieniu basic-auth)
```

2.8. Rozwiązywanie problemów

Objaw	Prawdopodobna przyczyna	Rozwiązanie
Backend nie uruchamia się	brak kluczy RSA	Wykonaj krok 2.3
`connection refused` na porcie 5432	PostgreSQL nie gotowy	`docker compose logs postgres` — sprawdź healthcheck
Migracje się nie aplikują	wyścig blokad	Sprawdź logi: `docker compose logs backend | grep alembic`
Celery nie pobiera zadań	zły `RABBITMQ_URL`	Porównaj hasło w `.env` z `RABBITMQ_PASSWORD`
MFA rzuca błąd szyfrowania	brak `ENCRYPTION_KEY`	Wygeneruj klucz Fernet (krok 2.3)
MinIO `connection refused`	healthcheck w toku	Odczekaj 30 s; `docker compose logs minio`
Port zajęty	konflikt z lokalną usługą	Zmień `POSTGRES_HOST_PORT` w `.env` lub zatrzymaj kolidującą usługę
Brak uprawnień do pliku	`docker-entrypoint.sh` bez `+x`	`chmod +x backend/docker-entrypoint.sh`

3. Instalacja multi-node (Kubernetes)

> **Uwaga:** Manifesty Kubernetes (Helm chart) zostaną dostarczone w katalogu `infrastructure/helm/testgen/`. Poniższa dokumentacja opisuje docelową architekturę wdrożenia.

3.1. Wymagania klastra

Wymaganie	Minimalna wersja	Uwagi
Kubernetes	1.28+	zarządzany (EKS / GKE / AKS) lub bare-metal (kubeadm)
Helm	3.12+	menedżer pakietów
Sterownik CSI	wymagany	obsługa PersistentVolumes (np. EBS CSI, Ceph)
Wtyczka CNI	wymagana	Calico, Cilium lub Flannel
Kontroler Ingress	wymagany	nginx-ingress lub Traefik
cert-manager	1.12+	automatyzacja certyfikatów TLS

3.2. Struktura Helm chart

```
infrastructure/helm/testgen/
├── Chart.yaml
├── values.yaml              # domyślna konfiguracja
├── values-production.yaml   # nadpisania produkcyjne
├── templates/
│   ├── _helpers.tpl
│   ├── namespace.yaml
│   ├── configmap.yaml
│   ├── secret.yaml
│   ├── deployment-backend.yaml
│   ├── deployment-frontend.yaml
│   ├── deployment-celery-worker.yaml
│   ├── deployment-celery-beat.yaml
│   ├── deployment-flower.yaml
│   ├── statefulset-postgres.yaml
│   ├── statefulset-redis.yaml
│   ├── statefulset-rabbitmq.yaml
│   ├── statefulset-minio.yaml
│   ├── service-backend.yaml
│   ├── service-frontend.yaml
│   ├── service-postgres.yaml
│   ├── service-redis.yaml
│   ├── service-rabbitmq.yaml
│   ├── service-minio.yaml
│   ├── ingress.yaml
│   ├── hpa-backend.yaml
│   ├── hpa-celery-worker.yaml
│   ├── pvc-postgres.yaml
│   ├── pvc-redis.yaml
│   ├── pvc-rabbitmq.yaml
│   └── pvc-minio.yaml
└── charts/                  # zależności (opcjonalne subcharty)
```

Kluczowe parametry `values.yaml` — patrz `examples/values.yaml`.

3.3. Przestrzeń nazw i RBAC

```bash
# Utworzenie przestrzeni nazw
kubectl create namespace testgen

# Ustawienie kontekstu
kubectl config set-context --current --namespace=testgen
```

Zalecane konto serwisowe z minimalnymi uprawnieniami:

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: testgen-app
  namespace: testgen
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: testgen-app-role
  namespace: testgen
rules:
  - apiGroups: [""]
    resources: ["configmaps", "secrets"]
    verbs: ["get", "list"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: testgen-app-binding
  namespace: testgen
subjects:
  - kind: ServiceAccount
    name: testgen-app
roleRef:
  kind: Role
  name: testgen-app-role
  apiGroup: rbac.authorization.k8s.io
```

3.4. Zarządzanie sekretami

⚠️ **Nigdy nie przechowuj sekretów w `values.yaml` w repozytorium.**

Opcja A — Sealed Secrets (Bitnami):

```bash
# Instalacja kontrolera
helm repo add sealed-secrets https://bitnami-labs.github.io/sealed-
         secrets
helm install sealed-secrets sealed-secrets/sealed-secrets -n kube-system

# Szyfrowanie sekretu
kubectl create secret generic testgen-secrets \
  --namespace=testgen \
  --from-literal=POSTGRES_PASSWORD='<silne_hasło>' \
  --from-literal=RABBITMQ_PASSWORD='<silne_hasło>' \
  --from-literal=MINIO_ROOT_PASSWORD='<silne_hasło>' \
  --from-literal=SECRET_KEY='<losowy_ciąg_64>' \
  --from-literal=ENCRYPTION_KEY='<klucz_fernet>' \
  --from-file=JWT_PRIVATE_KEY=backend/keys/private.pem \
  --from-file=JWT_PUBLIC_KEY=backend/keys/public.pem \
  --dry-run=client -o yaml | kubeseal -o yaml > sealed-secret.yaml

kubectl apply -f sealed-secret.yaml
```

Opcja B — HashiCorp Vault:

```
# Agent Injector – adnotacje w deployment:
# vault.hashicorp.com/agent-inject: "true"
# vault.hashicorp.com/role: "testgen"
# vault.hashicorp.com/agent-inject-secret-db:
        "secret/data/testgen/postgres"
```

3.5. Wolumeny trwałe (PV / PVC)

Komponent	Rozmiar PVC	Klasa magazynu	Tryb dostępu
PostgreSQL	100 Gi	`ssd` (gp3 / premium-rwo)	ReadWriteOnce
Redis	10 Gi	`ssd`	ReadWriteOnce
RabbitMQ	20 Gi	`ssd`	ReadWriteOnce
MinIO	500 Gi+	`standard` (gp2 / standard-rwo)	ReadWriteOnce

Przykład PVC dla PostgreSQL:

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: testgen-postgres-data
  namespace: testgen
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: ssd
  resources:
    requests:
      storage: 100Gi
```

3.6. Obiekt Ingress i routing

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: testgen-ingress
  namespace: testgen
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt-prod
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
    nginx.ingress.kubernetes.io/proxy-body-size: "500m"
spec:
  ingressClassName: nginx
  tls:
    - hosts:
        - testgen.example.com
      secretName: testgen-tls
  rules:
    - host: testgen.example.com
      http:
        paths:
          - path: /api/
            pathType: Prefix
            backend:
              service:
                name: testgen-backend
                port:
                  number: 8000
          - path: /
            pathType: Prefix
            backend:
              service:
                name: testgen-frontend
                port:
                  number: 80
```

3.7. Instalacja za pomocą Helm

```bash
# Dodanie repozytorium (jeśli chart jest hostowany)
# helm repo add testgen https://charts.example.com/testgen

# Instalacja z pliku lokalnego
helm install testgen ./infrastructure/helm/testgen \
  --namespace testgen \
  --values ./infrastructure/helm/testgen/values-production.yaml \
  --set secrets.existingSecret=testgen-secrets \
  --wait --timeout 10m

# Weryfikacja
kubectl get pods -n testgen
kubectl get svc -n testgen
kubectl get ingress -n testgen
```

3.8. Automatyczne skalowanie (HPA)

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: testgen-backend-hpa
  namespace: testgen
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: testgen-backend
  minReplicas: 2
  maxReplicas: 8
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 80
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: testgen-celery-worker-hpa
  namespace: testgen
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: testgen-celery-worker
  minReplicas: 2
  maxReplicas: 6
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 75
```

3.9. Weryfikacja wdrożenia

```
# 1. Wszystkie pody w stanie Running
kubectl get pods -n testgen
# NAME                                READY   STATUS    RESTARTS
        AGE
# testgen-backend-xxxxx               1/1     Running   0          5m
# testgen-frontend-xxxxx              1/1     Running   0          5m
# testgen-celery-worker-xxxxx         1/1     Running   0          5m
# testgen-celery-beat-xxxxx           1/1     Running   0          5m
# testgen-postgres-0                  1/1     Running   0          5m
# testgen-redis-0                     1/1     Running   0          5m
# testgen-rabbitmq-0                  1/1     Running   0          5m
# testgen-minio-0                     1/1     Running   0          5m

# 2. Healthcheck backendu
kubectl exec -n testgen deploy/testgen-backend -- \
  curl -sf http://localhost:8000/api/v1/health

# 3. Status Ingress
kubectl describe ingress testgen-ingress -n testgen

# 4. Certyfikat TLS
kubectl get certificate -n testgen
# NAME          READY   SECRET        AGE
# testgen-tls   True    testgen-tls   5m

# 5. Logi aplikacji
kubectl logs -n testgen deploy/testgen-backend --tail=50
kubectl logs -n testgen deploy/testgen-celery-worker --tail=50
```

---

4. Konfiguracja SSL/TLS

4.1. Let's Encrypt z cert-manager

Instalacja cert-manager (jeśli nie zainstalowany):

```
helm repo add jetstack https://charts.jetstack.io
helm install cert-manager jetstack/cert-manager \
  --namespace cert-manager --create-namespace \
  --set crds.enabled=true
```

ClusterIssuer dla Let's Encrypt:

```yaml
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt-prod
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    email: devops@example.com
    privateKeySecretRef:
      name: letsencrypt-prod-key
    solvers:
      - http01:
          ingress:
            ingressClassName: nginx
```

Certyfikat jest automatycznie odnawiany 30 dni przed wygaśnięciem.

4.2. Własny urząd certyfikacji (CA)

Dla środowisk bez dostępu do internetu (air-gapped):

```bash
# 1. Generowanie klucza CA
openssl genrsa -out ca.key 4096

# 2. Certyfikat CA (ważność 10 lat)
openssl req -x509 -new -nodes -key ca.key -sha256 \
  -days 3650 -out ca.crt \
  -subj "/C=PL/ST=Mazowieckie/O=TestGEN/CN=TestGEN CA"

# 3. Klucz serwera
openssl genrsa -out server.key 2048

# 4. Żądanie podpisu (CSR)
openssl req -new -key server.key -out server.csr \
  -subj "/C=PL/ST=Mazowieckie/O=TestGEN/CN=testgen.example.com"

# 5. Podpisanie certyfikatu serwera
openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key \
  -CAcreateserial -out server.crt -days 365 -sha256

# 6. Import do Kubernetes
kubectl create secret tls testgen-tls \
  --cert=server.crt --key=server.key \
  --namespace=testgen
```

4.3. Terminacja HTTPS

Single-node — nginx jako reverse proxy:

Utwórz plik `/etc/nginx/sites-available/testgen`:

```
server {
    listen 80;
    server_name testgen.example.com;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name testgen.example.com;

    ssl_certificate     /etc/ssl/certs/testgen.crt;
    ssl_certificate_key /etc/ssl/private/testgen.key;
    ssl_protocols       TLSv1.2 TLSv1.3;
    ssl_ciphers         HIGH:!aNULL:!MD5;

    # Nagłówki bezpieczeństwa
    add_header Strict-Transport-Security "max-age=31536000;
includeSubDomains" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-Frame-Options "SAMEORIGIN" always;

    # Maksymalny rozmiar uploadu (500 MB dla planu Professional)
    client_max_body_size 500m;

    # Frontend
    location / {
        proxy_pass http://127.0.0.1:4200;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # Backend API
    location /api/ {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        # Timeouty dla długich analiz
        proxy_read_timeout 600s;
        proxy_send_timeout 600s;
    }

    # Flower (opcjonalnie – ograniczenie IP)
    location /flower/ {
        allow 10.0.0.0/8;
        deny all;
        proxy_pass http://127.0.0.1:5555/;
    }
}
```

```
sudo ln -s /etc/nginx/sites-available/testgen /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
```

Kubernetes — terminacja TLS realizowana jest przez Ingress (sekcja 3.6).

---

5. Konfiguracja bazy danych (PostgreSQL)

5.1. Instalacja i inicjalizacja

W trybie Docker Compose PostgreSQL uruchamia się automatycznie z obrazu `postgres:16-alpine`. Konfiguracja wstępna odbywa się przez zmienne środowiskowe w pliku `.env`.

⚠️ W przypadku instalacji bare-metal:

```
# AlmaLinux
sudo dnf install -y postgresql16-server postgresql16
sudo /usr/pgsql-16/bin/postgresql-16-setup initdb
sudo systemctl enable --now postgresql-16

# Ubuntu
sudo apt install -y postgresql-16
sudo systemctl enable --now postgresql
```

5.2. Dostrajanie `postgresql.conf`

Plik przykładowy: `examples/postgresql.conf`.

Kluczowe parametry dla maszyny 8 GB RAM / 4 vCPU:

```
# Pamięć
shared_buffers = 2GB                    # 25% RAM
effective_cache_size = 6GB              # 75% RAM
work_mem = 64MB                         # per operacja sortowania
maintenance_work_mem = 512MB            # VACUUM, CREATE INDEX

# WAL i checkpoint
wal_buffers = 64MB
checkpoint_completion_target = 0.9
min_wal_size = 1GB
max_wal_size = 4GB

# Połączenia
max_connections = 200                   # z PgBouncer: może być niższe
listen_addresses = '*'                  # w Docker: 0.0.0.0

# Logowanie
log_min_duration_statement = 500        # ms — logi wolnych zapytań
log_checkpoints = on
log_connections = on
log_disconnections = on
log_lock_waits = on

# Replikacja (przygotowanie do backup PITR)
wal_level = replica
archive_mode = on
archive_command = 'cp %p /var/lib/postgresql/archive/%f'
```

Zastosowanie konfiguracji:

```
# Docker Compose — podmontowanie pliku:
# volumes:
#   - ./examples/postgresql.conf:/etc/postgresql/postgresql.conf:ro
# command: postgres -c config_file=/etc/postgresql/postgresql.conf

# Weryfikacja
docker compose exec postgres psql -U testgen -c "SHOW shared_buffers;"
```

5.3. Row-Level Security (RLS)

TestGEN używa RLS do izolacji danych między tenantami. Polityki są tworzone automatycznie przez migracje Alembic.

Weryfikacja:

```
docker compose exec postgres psql -U testgen -d testgen -c "
  SELECT schemaname, tablename, policyname, permissive, cmd
  FROM pg_policies
  ORDER BY tablename;
"
```

Oczekiwany wynik — polityki `tenant_isolation_policy` na każdej tabeli z kolumną `tenant_id`.

Testowanie izolacji:

```
-- Ustawienie kontekstu tenanta (symulacja aplikacji)
SET app.current_tenant = 'id-tenanta-A';
SELECT * FROM projects;  -- widoczne tylko projekty tenanta A

SET app.current_tenant = 'id-tenanta-B';
SELECT * FROM projects;  -- widoczne tylko projekty tenanta B
```

5.4. Rozszerzenia: pgcrypto i uuid-ossp

Rozszerzenia są tworzone automatycznie przez migracje. Weryfikacja:

```
docker compose exec postgres psql -U testgen -d testgen -c "
  SELECT extname, extversion FROM pg_extension
  WHERE extname IN ('pgcrypto', 'uuid-ossp');
"
# Oczekiwane:
#   pgcrypto  | 1.3
#   uuid-ossp | 1.1
```

5.5. PgBouncer — pula połączeń

Zalecany w produkcji dla ograniczenia liczby bezpośrednich połączeń do PostgreSQL.

Plik konfiguracyjny: `examples/pgbouncer.ini`.

```
[databases]
testgen = host=postgres port=5432 dbname=testgen

[pgbouncer]
listen_addr = 0.0.0.0
listen_port = 6432
auth_type = scram-sha-256
auth_file = /etc/pgbouncer/userlist.txt

# Tryb puli — transaction (zalecany dla async SQLAlchemy)
pool_mode = transaction
max_client_conn = 500
default_pool_size = 25
min_pool_size = 5
reserve_pool_size = 5
reserve_pool_timeout = 3

# Timeouty
server_idle_timeout = 300
client_idle_timeout = 600

# Logowanie
log_connections = 1
log_disconnections = 1
stats_period = 60
```

Plik `userlist.txt`:

```
"testgen" "<hasło_md5_lub_scram>"
```

Po włączeniu PgBouncer zmień `DATABASE_URL` w `.env`:

```
DATABASE_URL=postgresql+asyncpg://testgen:<hasło>@pgbouncer:6432/testgen
```

5.6. Migracje (Alembic)

Migracje uruchamiają się automatycznie przy starcie kontenera `backend` (`docker-entrypoint.sh`).

Ręczne zarządzanie:

```
# Status bieżący
docker compose exec backend alembic current

# Zastosowanie wszystkich migracji
docker compose exec backend alembic upgrade head

# Cofnięcie ostatniej migracji
docker compose exec backend alembic downgrade -1

# Historia migracji
docker compose exec backend alembic history --verbose | head -30

# Weryfikacja tabel
docker compose exec postgres psql -U testgen -d testgen -c "\dt"
```

---

6. Konfiguracja Redis

6.1. Tryb pracy i persistence

TestGEN wykorzystuje Redis do dwóch celów:

- **DB 0** — pamięć podręczna aplikacji (cache sesji, limitów)
- **DB 1** — wyniki zadań Celery (`CELERY_RESULT_BACKEND`)

Plik konfiguracyjny: `examples/redis.conf`.

```
# Pamięć
maxmemory 4gb

# Persistence — RDB + AOF (zalecane w produkcji)
save 900 1          # zapis co 15 min jeśli ≥ 1 zmiana
save 300 10         # zapis co 5 min jeśli ≥ 10 zmian
save 60 10000       # zapis co 1 min jeśli ≥ 10000 zmian

appendonly yes
appendfsync everysec

# Bezpieczeństwo
requirepass <silne_hasło>
rename-command FLUSHALL ""
rename-command FLUSHDB ""
rename-command CONFIG ""

# Sieć
bind 0.0.0.0
protected-mode yes
```

Zastosowanie w Docker Compose:

```
redis:
  image: redis:7-alpine
  command: redis-server /usr/local/etc/redis/redis.conf
  volumes:
    - redis-data:/data
    - ./examples/redis.conf:/usr/local/etc/redis/redis.conf:ro
```

6.2. Polityka eksmisji

Polityka	Kiedy stosować
`allkeys-lru`	**zalecana** — usuwa najdawniej nieużywane klucze
`volatile-lru`	tylko klucze z TTL — bezpieczne, ale wymaga TTL na wszystkich kluczach
`noeviction`	zwraca błąd przy braku pamięci — do debugowania

```
maxmemory-policy allkeys-lru
```

6.3. Weryfikacja

```
# Ping
docker compose exec redis redis-cli ping
# PONG

# Informacje o pamięci
docker compose exec redis redis-cli INFO memory | grep used_memory_human

# Informacje o persistence
docker compose exec redis redis-cli INFO persistence | grep -E
        "rdb_|aof_"

# Test zapisu/odczytu
docker compose exec redis redis-cli SET test "ok" EX 10
docker compose exec redis redis-cli GET test
# "ok"
```

---

7. Konfiguracja RabbitMQ

7.1. Kolejki i exchange

TestGEN wykorzystuje RabbitMQ jako broker Celery. Kolejki są tworzone automatycznie przy starcie workerów.

Kolejka	Przeznaczenie	Priorytet
`default`	zadania ogólne (email, cleanup)	normalny
`analysis`	analizy kodu/dokumentacji/UML/GUI	wysoki
`export`	eksport raportów (PDF, DOCX, CSV)	normalny

Kolejka	Przeznaczenie	Priorytet
`security`	skanowanie bezpieczeństwa	wysoki

Plik konfiguracyjny: `examples/rabbitmq.conf`.

```
# Limity pamięci
vm_memory_high_watermark.relative = 0.6
vm_memory_high_watermark_paging_ratio = 0.8
disk_free_limit.relative = 1.5

# Logowanie
log.console = true
log.console.level = warning
log.file.level = info

# Management plugin (UI na porcie 15672)
management.tcp.port = 15672

# Limity połączeń
channel_max = 128
heartbeat = 60
```

7.2. Wirtualne hosty

Zalecana izolacja środowisk (dev / staging / produkcja) za pomocą vhostów:

```
# Utworzenie vhosta produkcyjnego
docker compose exec rabbitmq rabbitmqctl add_vhost /production

# Uprawnienia użytkownika
docker compose exec rabbitmq rabbitmqctl set_permissions \
  -p /production testgen ".*" ".*" ".*"
```

W `.env` zmień URL brokera:

```
RABBITMQ_URL=amqp://testgen:<hasło>@rabbitmq:5672/production
CELERY_BROKER_URL=amqp://testgen:<hasło>@rabbitmq:5672/production
```

7.3. Wysoka dostępność (HA)

Dla klastrów Kubernetes z wieloma replikami RabbitMQ:

```
# Polityka HA — mirroring kolejek (classic mirroring)
docker compose exec rabbitmq rabbitmqctl set_policy ha-all \
  "^" '{"ha-mode":"all","ha-sync-mode":"automatic"}' \
  --apply-to queues

# Lub w RabbitMQ 3.13+ — Quorum Queues (zalecane):
# Celery konfiguracja:
# CELERY_BROKER_TRANSPORT_OPTIONS='{"queue_arguments": {"x-queue-type":
        "quorum"}}'
```

7.4. Weryfikacja

```
# Status serwera
docker compose exec rabbitmq rabbitmqctl status | head -30

# Lista kolejek
docker compose exec rabbitmq rabbitmqctl list_queues \
  name messages consumers state

# Lista vhostów
docker compose exec rabbitmq rabbitmqctl list_vhosts

# Lista połączeń
docker compose exec rabbitmq rabbitmqctl list_connections \
  user vhost state

# Panel zarządzania (przeglądarka)
# http://localhost:15672  (login: testgen / hasło z .env)
```

---

8. Konfiguracja MinIO / S3

8.1. Tworzenie zasobników (buckets)

TestGEN wymaga następujących zasobników:

Zasobnik	Przeznaczenie	Retencja
`testgen-uploads`	pliki źródłowe (kod, dokumentacja, UML, GUI)	usuwane po analizie (FR9)
`testgen-results`	wyniki analiz, raporty	długoterminowa
`testgen-exports`	wyeksportowane pliki (PDF, DOCX)	30 dni
`testgen-avatars`	awatary użytkowników	długoterminowa

```
# Instalacja klienta mc
docker compose exec minio mc alias set local http://localhost:9000 \
  ${MINIO_ROOT_USER} ${MINIO_ROOT_PASSWORD}

# Tworzenie zasobników
docker compose exec minio mc mb local/testgen-uploads
docker compose exec minio mc mb local/testgen-results
docker compose exec minio mc mb local/testgen-exports
docker compose exec minio mc mb local/testgen-avatars
```

8.2. Polityki dostępu

```
# Polityka tylko do odczytu dla zasobnika exports (dostęp publiczny do
    pobrania)
docker compose exec minio mc anonymous set download local/testgen-exports

# Polityka prywatna (domyślna) dla reszty
docker compose exec minio mc anonymous set none local/testgen-uploads
docker compose exec minio mc anonymous set none local/testgen-results
```

Dedykowany użytkownik serwisowy (zamiast root):

```
# Utworzenie użytkownika aplikacji
docker compose exec minio mc admin user add local testgen-app
    <silne_hasło>

# Polityka z pełnym dostępem do zasobników testgen-*
cat <<'EOF' > /tmp/testgen-policy.json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:*"],
      "Resource": [
        "arn:aws:s3:::testgen-*",
        "arn:aws:s3:::testgen-*/*"
      ]
    }
  ]
}
EOF

docker compose exec minio mc admin policy create local testgen-policy
    /tmp/testgen-policy.json
docker compose exec minio mc admin policy attach local testgen-policy --
    user testgen-app
```

8.3. Reguły cyklu życia

```
# Automatyczne usuwanie plików z testgen-exports po 30 dniach
docker compose exec minio mc ilm rule add local/testgen-exports \
  --expire-days 30

# Automatyczne usuwanie plików z testgen-uploads po 7 dniach
# (backup — główne czyszczenie realizuje Celery task cleanup)
docker compose exec minio mc ilm rule add local/testgen-uploads \
  --expire-days 7

# Weryfikacja reguł
docker compose exec minio mc ilm rule list local/testgen-uploads
docker compose exec minio mc ilm rule list local/testgen-exports
```

8.4. Weryfikacja

```
# Informacje o serwerze
docker compose exec minio mc admin info local

# Lista zasobników
docker compose exec minio mc ls local/
# Oczekiwane:
# [2026-09-16 10:00:00 UTC]     0B testgen-uploads/
# [2026-09-16 10:00:00 UTC]     0B testgen-results/
# [2026-09-16 10:00:00 UTC]     0B testgen-exports/
# [2026-09-16 10:00:00 UTC]     0B testgen-avatars/

# Konsola webowa MinIO
# http://localhost:9001 (login: MINIO_ROOT_USER / MINIO_ROOT_PASSWORD)

# Test uploadu
echo "test" | docker compose exec -T minio mc pipe local/testgen-
         uploads/test.txt
docker compose exec minio mc cat local/testgen-uploads/test.txt
docker compose exec minio mc rm local/testgen-uploads/test.txt
```

Powiązane dokumenty

Dokument	Opis
`getting-started.md`	szybki start dla programistów
`environment-setup.md`	pełna lista zmiennych środowiskowych
`02-operacje.md`	operacje, kopie zapasowe, skalowanie (Story 9.7)
`examples/`	przykładowe pliki konfiguracyjne

Podręcznik operacyjny — backup, skalowanie, aktualizacje, monitoring, DR

> **Wersja:** 1.0
> **Data:** 2026-09-16
> **Docelowy czytelnik:** inżynier DevOps / administrator systemów
> **Dokument uzupełniający:** 01-instalacja.md

Spis treści

1. Kopie zapasowe i przywracanie
    1. Strategia kopii zapasowych
    2. PostgreSQL — pg_dump i WAL archiving
    3. Redis — RDB i AOF
    4. MinIO — replikacja zasobników
    5. Konfiguracja i sekrety
    6. Procedura przywracania
    7. Checklist kopii zapasowych
2. Skalowanie horyzontalne
    1. Skalowanie workerów Celery
    2. Skalowanie backendu FastAPI
    3. Skalowanie PostgreSQL — repliki odczytu
    4. Skalowanie Redis
    5. Automatyczne skalowanie (K8s HPA)
    6. Tabela skalowania
    7. Checklist skalowania
3. Aktualizacje systemu
    1. Przygotowanie do aktualizacji
    2. Aktualizacja Docker Compose
    3. Aktualizacja Kubernetes
    4. Migracje bazy danych (Alembic)
    5. Procedura wycofania zmian (rollback)
    6. Checklist aktualizacji
4. Monitoring
    1. Architektura stosu monitoringu
    2. Konfiguracja Prometheus
    3. Konfiguracja Grafana
    4. Konfiguracja Loki i Promtail
    5. Reguły alertów
    6. Alertmanager — routing i eskalacja
5. Odtwarzanie po awarii (DR)
    1. Strategia DR
    2. Procedura odtwarzania

3. Ośrodek zapasowy (DR site)
4. Test DR — procedura kwartalna
5. Checklist DR

Format procedur

Każda procedura jest opisana według schematu:

Element	Opis
Cel	co procedura realizuje
Wymagania wstępne	co musi być spełnione przed rozpoczęciem
Kroki	numerowana lista poleceń z oczekiwanymi wynikami
Weryfikacja	jak potwierdzić poprawność wykonania
Czas wykonania	szacowany czas
Checklist	lista kontrolna do odznaczenia

---

1. Kopie zapasowe i przywracanie

1.1. Strategia kopii zapasowych

Parametr	Wartość	Źródło
RPO (maksymalna utrata danych)	< 1 godzina	NFR32
RTO (maksymalny czas przywracania)	< 4 godziny	NFR31
Retencja kopii dziennych	30 dni	—
Retencja kopii tygodniowych	12 tygodni	—
Retencja kopii miesięcznych	12 miesięcy	—

Harmonogram:

Komponent	Typ kopii	Częstotliwość	Metoda
PostgreSQL	pełna (pg_dump)	codziennie 02:00	skrypt + cron
PostgreSQL	ciągła (WAL)	ciągle	archive_command
Redis	migawka (RDB)	co 15 min / co 5 min	konfiguracja save
Redis	dziennik (AOF)	ciągle	appendfsync everysec
MinIO	replikacja	codziennie 03:00	mc mirror
Konfiguracja	pełna	po każdej zmianie	skrypt + git

Miejsce docelowe kopii:

- Magazyn zdalny (S3-compatible, NFS, lub drugi serwer)
- ⚠️ **Nigdy** nie przechowuj kopii na tym samym dysku co dane źródłowe

1.2. PostgreSQL — pg_dump i WAL archiving

Pełna kopia (pg_dump)

Cel: kompletna kopia bazy danych z możliwością przywrócenia na czystej instancji.

Wymagania wstępne: dostęp do kontenera `postgres`, katalog docelowy z wystarczającą przestrzenią.

Kroki:

```
# 1. Utworzenie katalogu na kopie
mkdir -p /backup/postgresql/daily

# 2. Wykonanie kopii w formacie custom (kompresja + selektywne
        przywracanie)
docker compose exec -T postgres pg_dump \
  -U testgen \
  -d testgen \
  -Fc \
  --verbose \
  > /backup/postgresql/daily/testgen_$(date +%Y%m%d_%H%M%S).dump

# 3. Sprawdzenie rozmiaru kopii
ls -lh /backup/postgresql/daily/testgen_*.dump | tail -1
```

Weryfikacja:

```
# Sprawdzenie integralności kopii
pg_restore --list /backup/postgresql/daily/testgen_XXXXXXXX_XXXXXX.dump |
        head -20
# Powinien wyświetlić listę obiektów (tabele, indeksy, sekwencje)
```

Czas wykonania: 2–15 min (zależnie od rozmiaru bazy; ~1 GB/min).

Skrypt automatyzujący: `examples/backup-postgres.sh`.

Ciągła kopia (WAL archiving + PITR)

Cel: możliwość przywrócenia bazy do dowolnego punktu w czasie (RPO bliskie zeru).

Wymagania wstępne: `postgresql.conf` z włączonym `archive_mode` i `wal_level = replica` (patrz 01-instalacja.md, sekcja 5.2).

Konfiguracja:

```
# postgresql.conf
wal_level = replica
archive_mode = on
archive_command = 'cp %p /backup/postgresql/wal/%f'
archive_timeout = 300    # wymuś archiwizację co 5 min (= RPO ≤ 5 min)
```

```
# Utworzenie katalogu WAL
mkdir -p /backup/postgresql/wal

# Wykonanie base backup (punkt startowy PITR)
docker compose exec -T postgres pg_basebackup \
  -U testgen \
  -D /tmp/basebackup \
  --format=tar \
  --gzip \
  --checkpoint=fast \
  --label="basebackup_$(date +%Y%m%d)"

# Skopiowanie na zewnątrz kontenera
docker compose cp postgres:/tmp/basebackup
         /backup/postgresql/basebackup_$(date +%Y%m%d)
```

Przywracanie PITR:

```
# 1. Zatrzymanie PostgreSQL
docker compose stop postgres

# 2. Rozpakowanie base backup
rm -rf /var/lib/postgresql/data/*
tar xzf /backup/postgresql/basebackup_XXXXXXXX/base.tar.gz -C
         /var/lib/postgresql/data/

# 3. Konfiguracja przywracania (recovery.conf / postgresql.conf w PG16)
cat >> /var/lib/postgresql/data/postgresql.conf << 'EOF'
restore_command = 'cp /backup/postgresql/wal/%f %p'
recovery_target_time = '2026-09-16 14:30:00+02'   # punkt docelowy
recovery_target_action = promote
EOF

# 4. Utworzenie sygnału przywracania
touch /var/lib/postgresql/data/recovery.signal

# 5. Uruchomienie — PG przywróci dane do wskazanego punktu
docker compose start postgres

# 6. Weryfikacja
docker compose exec postgres psql -U testgen -c "SELECT
         pg_is_in_recovery();"
# Oczekiwane: false (po zakończeniu przywracania)
```

1.3. Redis — RDB i AOF

Kopia RDB (migawka):

```
# Ręczne wyzwolenie migawki
docker compose exec redis redis-cli BGSAVE
# Background saving started

# Sprawdzenie statusu
docker compose exec redis redis-cli LASTSAVE
# Unix timestamp ostatniej migawki

# Skopiowanie pliku RDB
docker compose cp redis:/data/dump.rdb /backup/redis/dump_$(date
        +%Y%m%d_%H%M%S).rdb
```

Kopia AOF (dziennik):

```
# Ręczne wymuszenie przepisania AOF
docker compose exec redis redis-cli BGREWRITEAOF

# Skopiowanie pliku AOF
docker compose cp redis:/data/appendonly.aof
        /backup/redis/appendonly_$(date +%Y%m%d).aof
```

Przywracanie Redis:

```
# 1. Zatrzymanie Redis
docker compose stop redis

# 2. Podmiana pliku danych
docker compose cp /backup/redis/dump_XXXXXXXX.rdb redis:/data/dump.rdb
# lub:
docker compose cp /backup/redis/appendonly_XXXXXXXX.aof
        redis:/data/appendonly.aof

# 3. Uruchomienie
docker compose start redis

# 4. Weryfikacja
docker compose exec redis redis-cli DBSIZE
```

Czas wykonania: < 1 min (RDB), < 5 min (AOF rewrite).

Skrypt automatyzujący: `examples/backup-redis.sh`.

1.4. MinIO — replikacja zasobników

Cel: kopia wszystkich plików przechowywanych w MinIO (kod źródłowy, dokumentacja, wyniki analiz).

```
# 1. Konfiguracja aliasu docelowego (zdalny serwer lub drugi MinIO)
docker compose exec minio mc alias set backup https://backup-
        minio.example.com \
  <BACKUP_ACCESS_KEY> <BACKUP_SECRET_KEY>

# 2. Replikacja zasobników
docker compose exec minio mc mirror local/testgen-results backup/testgen-
        results
docker compose exec minio mc mirror local/testgen-exports backup/testgen-
        exports
docker compose exec minio mc mirror local/testgen-avatars backup/testgen-
        avatars
# testgen-uploads — opcjonalnie (pliki usuwane po analizie, FR9)

# 3. Weryfikacja
docker compose exec minio mc ls backup/
docker compose exec minio mc du backup/testgen-results
```

Alternatywa — kopia lokalna:

```
# Kopia na dysk lokalny
mkdir -p /backup/minio
docker compose exec minio mc mirror local/testgen-results
        /backup/minio/testgen-results
```

Czas wykonania: zmienny, zależnie od ilości danych (5–60 min).

Skrypt automatyzujący: `examples/backup-minio.sh`.

1.5. Konfiguracja i sekrety

⚠️ Sekrety wymagają osobnego zabezpieczenia — szyfrowane kopie lub sejf haseł.

```
# Katalog docelowy
mkdir -p /backup/config

# Pliki konfiguracyjne
cp .env /backup/config/.env_$(date +%Y%m%d)
cp docker-compose.yml /backup/config/
cp docker-compose.dev.yml /backup/config/

# Klucze JWT (krytyczne!)
cp -r backend/keys/ /backup/config/keys_$(date +%Y%m%d)/

# Certyfikaty SSL/TLS
cp /etc/ssl/certs/testgen.crt /backup/config/
cp /etc/ssl/private/testgen.key /backup/config/

# Szyfrowanie kopii (zalecane)
tar czf - /backup/config/ | \
  openssl enc -aes-256-cbc -pbkdf2 -out /backup/config_encrypted_$(date
        +%Y%m%d).tar.gz.enc
```

1.6. Procedura przywracania

Cel: pełne odtworzenie systemu z kopii zapasowych.

Wymagania wstępne: czysta maszyna z zainstalowanym Docker i Docker Compose (patrz 01-instalacja.md, sekcja 2.1).

Kroki:

```
# 1. Klonowanie repozytorium
git clone https://bitbucket.org/nexio/testgen.git && cd testgen

# 2. Przywrócenie konfiguracji
cp /backup/config/.env_XXXXXXXX .env
cp -r /backup/config/keys_XXXXXXXX/ backend/keys/

# 3. Uruchomienie infrastruktury (bez backendu)
docker compose up -d postgres redis rabbitmq minio
# Czekaj na healthchecks
sleep 30

# 4. Przywrócenie PostgreSQL
docker compose exec -T postgres pg_restore \
  -U testgen -d testgen \
  --clean --if-exists \
  --no-owner --no-privileges \
  < /backup/postgresql/daily/testgen_XXXXXXXX.dump

# 5. Przywrócenie Redis (opcjonalne — dane efemeryczne)
docker compose cp /backup/redis/dump_XXXXXXXX.rdb redis:/data/dump.rdb
docker compose restart redis

# 6. Przywrócenie MinIO
docker compose exec minio mc mirror /backup/minio/testgen-results
         local/testgen-results
docker compose exec minio mc mirror /backup/minio/testgen-exports
         local/testgen-exports

# 7. Uruchomienie aplikacji
docker compose up -d

# 8. Weryfikacja
curl -s http://localhost:8000/api/v1/health | python3 -m json.tool
curl -s -o /dev/null -w "%{http_code}" http://localhost:4200
```

Czas wykonania: 30–120 min (zależnie od rozmiaru bazy i danych MinIO).

Skrypt automatyzujący: `examples/restore-postgres.sh`.

1.7. Checklist kopii zapasowych

Kopia dzienna:

- ☐ Kopia PostgreSQL (`pg_dump`) zakończona bez błędów

- [ ] Plik kopii > 0 bajtów, integralność sprawdzona (`pg_restore --list`)
- [ ] WAL archiving działa (sprawdź opóźnienie: `SELECT pg_last_wal_replay_lsn()`)
- [ ] Migawka Redis (RDB) utworzona
- [ ] Replikacja MinIO zakończona (`mc mirror` bez błędów)
- [ ] Kopia przeniesiona na magazyn zdalny
- [ ] Usunięto kopie starsze niż retencja (30 dni / 12 tygodni / 12 miesięcy)

Przywracanie (test miesięczny):

- [ ] Przywrócenie PostgreSQL na instancji testowej — dane kompletne
- [ ] Przywrócenie Redis — klucze obecne
- [ ] Przywrócenie MinIO — pliki dostępne
- [ ] Aplikacja uruchamia się i odpowiada na healthcheck
- [ ] Czas przywracania < 4h (RTO)

---

2. Skalowanie horyzontalne

2.1. Skalowanie workerów Celery

Docker Compose — dodanie workerów:

```
# Uruchomienie dodatkowego workera (np. 3 instancje)
docker compose up -d --scale celery-worker=3

# Weryfikacja
docker compose ps | grep celery-worker
# celery-worker-1   Running
# celery-worker-2   Running
# celery-worker-3   Running
```

Dostrajanie współbieżności (concurrency):

```
# Zmiana concurrency per worker (domyślnie: liczba CPU)
# W docker-compose.yml lub docker-compose.override.yml:
#   command: celery -A app.workers.celery_app worker
#            --loglevel=info
#            -Q default,analysis,export,security
#            --concurrency=4

# Dla zadań intensywnych obliczeniowo (analysis) — niższa współbieżność
# Dla zadań I/O-bound (export, email) — wyższa współbieżność
```

Dedykowane workerki per kolejka (zalecane w produkcji):

```yaml
# docker-compose.override.yml
services:
  celery-worker-analysis:
    build:
      context: ./backend
      target: production
    command: celery -A app.workers.celery_app worker
             --loglevel=info -Q analysis --concurrency=2
    env_file: .env
    depends_on:
      rabbitmq: { condition: service_healthy }
      redis: { condition: service_healthy }

  celery-worker-default:
    build:
      context: ./backend
      target: production
    command: celery -A app.workers.celery_app worker
             --loglevel=info -Q default,export --concurrency=4
    env_file: .env
    depends_on:
      rabbitmq: { condition: service_healthy }
      redis: { condition: service_healthy }

  celery-worker-security:
    build:
      context: ./backend
      target: production
    command: celery -A app.workers.celery_app worker
             --loglevel=info -Q security --concurrency=2
    env_file: .env
    depends_on:
      rabbitmq: { condition: service_healthy }
      redis: { condition: service_healthy }
```

2.2. Skalowanie backendu FastAPI

Docker Compose:

```bash
# Uruchomienie dodatkowych replik (za reverse proxy / load balancerem)
docker compose up -d --scale backend=3
```

⚠️ **Uwaga:** wbudowany healthcheck (`/api/v1/health`) działa niezależnie na każdej replice. Reverse proxy (nginx) powinien równoważyć ruch round-robin.

Konfiguracja nginx (upstream):

```
upstream testgen-backend {
    least_conn;
    server 127.0.0.1:8001;
    server 127.0.0.1:8002;
    server 127.0.0.1:8003;
}

server {
    # ...
    location /api/ {
        proxy_pass http://testgen-backend;
        # ...
    }
}
```

Kubernetes:

```
kubectl scale deployment testgen-backend --replicas=4 -n testgen
kubectl get pods -l app=testgen-backend -n testgen
```

2.3. Skalowanie PostgreSQL — repliki odczytu

Cel: rozłożenie obciążenia odczytu na repliki (streaming replication).

Konfiguracja serwera głównego (primary):

```
# postgresql.conf (primary)
wal_level = replica
max_wal_senders = 5
wal_keep_size = 1GB
```

```
# Utworzenie użytkownika replikacji
docker compose exec postgres psql -U testgen -c "
  CREATE ROLE replicator WITH REPLICATION LOGIN PASSWORD '<silne_hasło>';
"
```

```
# pg_hba.conf (primary) — dozwolone połączenia replikacji
host replication replicator 10.0.0.0/8 scram-sha-256
```

Konfiguracja repliki (standby):

```
# 1. Wykonanie base backup z serwera głównego
pg_basebackup -h primary-host -U replicator -D /var/lib/postgresql/data \
  --wal-method=stream --checkpoint=fast

# 2. Konfiguracja standby
cat >> /var/lib/postgresql/data/postgresql.conf << 'EOF'
primary_conninfo = 'host=primary-host port=5432 user=replicator password=
        <hasło>'
hot_standby = on
EOF

# 3. Sygnał standby
touch /var/lib/postgresql/data/standby.signal

# 4. Uruchomienie repliki
pg_ctl start -D /var/lib/postgresql/data
```

Weryfikacja:

```
# Na serwerze głównym
psql -U testgen -c "SELECT * FROM pg_stat_replication;"
# Powinien wyświetlić aktywną replikę

# Na replice
psql -U testgen -c "SELECT pg_is_in_recovery();"
# Oczekiwane: true
```

2.4. Skalowanie Redis

Redis Sentinel (wysoka dostępność):

```
# sentinel.conf
sentinel monitor testgen-redis redis-primary 6379 2
sentinel down-after-milliseconds testgen-redis 5000
sentinel failover-timeout testgen-redis 60000
sentinel parallel-syncs testgen-redis 1
sentinel auth-pass testgen-redis <hasło_redis>
```

```
# Uruchomienie 3 instancji Sentinel
redis-sentinel /etc/redis/sentinel.conf
```

Redis Cluster (shardowanie):

Zalecany dla dużej ilości danych (> 16 GB) lub bardzo wysokiej przepustowości.

```
# Utworzenie klastra 6-węzłowego (3 master + 3 repliki)
redis-cli --cluster create \
  redis-1:6379 redis-2:6379 redis-3:6379 \
  redis-4:6379 redis-5:6379 redis-6:6379 \
  --cluster-replicas 1

# Weryfikacja
redis-cli --cluster info redis-1:6379
```

2.5. Automatyczne skalowanie (K8s HPA)

Reguły skalowania zgodne z NFR22:

Metryka	Próg skalowania w górę	Próg skalowania w dół	Komponent
CPU	> 70%	< 30%	backend, celery-worker
Pamięć	> 80%	< 40%	backend
Głębokość kolejki	> 100 zadań	< 10 zadań	celery-worker

```
# Sprawdzenie aktualnego stanu HPA
kubectl get hpa -n testgen

# Ręczna zmiana limitów
kubectl patch hpa testgen-backend-hpa -n testgen \
  --type merge -p '{"spec":{"maxReplicas":10}}'

# Obserwacja skalowania w czasie rzeczywistym
kubectl get hpa -n testgen -w
```

Manifesty HPA zdefiniowane w 01-instalacja.md, sekcja 3.8.

2.6. Tabela skalowania

Komponent	Min replik	Max replik	Metryka wyzwalająca	Uwagi
backend (FastAPI)	2	8	CPU > 70%	bezstanowy, łatwo skalowalny
celery-worker	2	6	CPU > 75%	dedykowane kolejki
celery-beat	1	1	—	**dokładnie 1** (singleton)
flower	1	1	—	monitoring, nie wymaga skalowania
frontend (nginx)	2	4	CPU > 70%	statyczne pliki, niskie zużycie
PostgreSQL	1+N	1+3	odczyt > 80% CPU	1 primary + N replik odczytu

Komponent	Min replik	Max replik	Metryka wyzwalająca	Uwagi
Redis	1	3 (Sentinel)	pamięć > 80%	lub Redis Cluster
RabbitMQ	1	3 (klaster)	głębokość kolejki	quorum queues
MinIO	1	4 (distributed)	dysk > 80%	distributed mode

2.7. Checklist skalowania

- ☐ Zidentyfikowano wąskie gardło (CPU / pamięć / I/O / sieć)
- ☐ Sprawdzono metryki w Grafana przed skalowaniem
- ☐ Dodano repliki / zwiększono zasoby
- ☐ Zweryfikowano healthchecks nowych instancji
- ☐ Zweryfikowano równoważenie ruchu (load balancing)
- ☐ Zaktualizowano alerty monitoringu
- ☐ Udokumentowano zmianę w dzienniku operacyjnym

---

3. Aktualizacje systemu

3.1. Przygotowanie do aktualizacji

⚠️ **Każda aktualizacja wymaga:**

1. **Kopia zapasowa** — pełna kopia PostgreSQL + konfiguracji (sekcja 1)
2. **Okno serwisowe** — zaplanowane, skomunikowane użytkownikom
3. **Plan wycofania** — przetestowana procedura rollback
4. **Środowisko testowe** — aktualizacja najpierw na staging

Przed rozpoczęciem:

```
# 1. Wykonanie kopii zapasowej
./examples/backup-postgres.sh
./examples/backup-redis.sh
./examples/backup-minio.sh

# 2. Zanotowanie aktualnych wersji
docker compose exec backend python -c "import app;
         print(app.__version__)"
docker compose exec postgres psql -U testgen -c "SELECT version();"
docker compose exec backend alembic current

# 3. Pobranie nowej wersji kodu
git fetch origin
git log --oneline HEAD..origin/main | head -10
```

3.2. Aktualizacja Docker Compose

Cel: aktualizacja systemu na serwerze single-node.

Czas wykonania: 5–15 min (z krótką przerwą w dostępności).

```
# 1. Pobranie nowej wersji
cd /opt/testgen
git pull origin main

# 2. Zbudowanie nowych obrazów
docker compose build --no-cache

# 3. Zatrzymanie aplikacji (nie bazy!)
docker compose stop backend frontend celery-worker celery-beat flower

# 4. Uruchomienie z nowymi obrazami
# (migracje bazy uruchomią się automatycznie — docker-entrypoint.sh)
docker compose up -d

# 5. Śledzenie logów startu
docker compose logs -f backend --tail=50
# Szukaj: "Application startup complete"

# 6. Weryfikacja
curl -s http://localhost:8000/api/v1/health | python3 -m json.tool
docker compose ps
```

Oczekiwane wyniki:

- Wszystkie serwisy w stanie `healthy` / `running`
- Healthcheck zwraca `{"status": "ok"}`
- Migracje zastosowane bez błędów

3.3. Aktualizacja Kubernetes

Rolling update (domyślny)

```
# 1. Aktualizacja obrazów w rejestrze
docker build -t registry.example.com/testgen/backend:v2.1.0 ./backend
docker push registry.example.com/testgen/backend:v2.1.0

docker build -t registry.example.com/testgen/frontend:v2.1.0 ./frontend
docker push registry.example.com/testgen/frontend:v2.1.0

# 2. Aktualizacja Helm
helm upgrade testgen ./infrastructure/helm/testgen \
  --namespace testgen \
  --set backend.image.tag=v2.1.0 \
  --set frontend.image.tag=v2.1.0 \
  --set celeryWorker.image.tag=v2.1.0 \
  --wait --timeout 10m

# 3. Śledzenie postępu
kubectl rollout status deployment/testgen-backend -n testgen
kubectl rollout status deployment/testgen-frontend -n testgen
kubectl rollout status deployment/testgen-celery-worker -n testgen
```

Blue-green deployment

```
# 1. Wdrożenie nowej wersji jako "green"
helm install testgen-green ./infrastructure/helm/testgen \
  --namespace testgen \
  --set backend.image.tag=v2.1.0 \
  --set ingress.enabled=false

# 2. Weryfikacja "green"
kubectl port-forward -n testgen svc/testgen-green-backend 8001:8000
curl -s http://localhost:8001/api/v1/health

# 3. Przełączenie ruchu (zmiana Ingress)
kubectl patch ingress testgen-ingress -n testgen \
  --type merge -p '{"spec":{"rules":
        [{"host":"testgen.example.com","http":{"paths":
        [{"path":"/api/","pathType":"Prefix","backend":{"service":
        {"name":"testgen-green-backend","port":{"number":8000}}}},
        {"path":"/","pathType":"Prefix","backend":{"service":
        {"name":"testgen-green-frontend","port":{"number":80}}}}]}}]}}'

# 4. Usunięcie starej wersji (po weryfikacji)
helm uninstall testgen -n testgen
helm rename testgen-green testgen  # lub reinstall
```

Canary deployment

```
# 1. Wdrożenie canary (1 replika)
helm install testgen-canary ./infrastructure/helm/testgen \
  --namespace testgen \
  --set backend.replicaCount=1 \
  --set backend.image.tag=v2.1.0 \
  --set ingress.enabled=false

# 2. Konfiguracja podziału ruchu (nginx Ingress canary)
kubectl annotate ingress testgen-canary-ingress -n testgen \
  nginx.ingress.kubernetes.io/canary="true" \
  nginx.ingress.kubernetes.io/canary-weight="10"

# 3. Monitorowanie błędów canary (Grafana / Prometheus)
# Jeśli error rate < 1% przez 30 min → zwiększ wagę do 50% → 100%
# Jeśli error rate > 5% → rollback canary
```

3.4. Migracje bazy danych (Alembic)

Migracje uruchamiają się **automatycznie** przy starcie kontenera `backend`. W przypadku ręcznej kontroli:

```
# Sprawdzenie aktualnej rewizji
docker compose exec backend alembic current

# Sprawdzenie oczekujących migracji
docker compose exec backend alembic history --indicate-current | head -20

# Zastosowanie migracji
docker compose exec backend alembic upgrade head

# Weryfikacja
docker compose exec backend alembic current
# Powinna wyświetlić rewizję (head)
```

⚠️ **Kompatybilność migracji:**

- Migracje **muszą** być kompatybilne wstecz (backward-compatible) na czas rolling update
- Unikaj: usuwania kolumn, zmiany nazw, zmiany typów w jednym kroku
- Wzorzec: dodaj nową kolumnę → migruj dane → usuń starą (w osobnych wydaniach)

3.5. Procedura wycofania zmian (rollback)

Docker Compose

```
# 1. Zatrzymanie serwisów
docker compose stop backend frontend celery-worker celery-beat flower

# 2. Cofnięcie kodu do poprzedniej wersji
git checkout <poprzedni_commit>

# 3. Zbudowanie i uruchomienie
docker compose build
docker compose up -d

# 4. Jeśli potrzebne — cofnięcie migracji
docker compose exec backend alembic downgrade -1

# 5. Weryfikacja
curl -s http://localhost:8000/api/v1/health | python3 -m json.tool
```

Kubernetes

```
# Rollback do poprzedniej rewizji Helm
helm rollback testgen 0 -n testgen --wait

# Lub rollback konkretnego deploymentu
kubectl rollout undo deployment/testgen-backend -n testgen

# Weryfikacja
kubectl rollout status deployment/testgen-backend -n testgen
```

Czas wykonania: 5–10 min (Docker Compose), 2–5 min (K8s rollback).

3.6. Checklist aktualizacji

Przed aktualizacją:

- ☐ Kopia zapasowa PostgreSQL wykonana i zweryfikowana
- ☐ Kopia konfiguracji (.env, klucze) wykonana
- ☐ Okno serwisowe zaplanowane i skomunikowane
- ☐ Changelog / release notes przeczytane
- ☐ Aktualizacja przetestowana na staging
- ☐ Plan rollback przygotowany

W trakcie:

- ☐ Nowe obrazy zbudowane
- ☐ Migracje zastosowane bez błędów
- ☐ Healthchecks przechodzą
- ☐ Logi bez błędów krytycznych

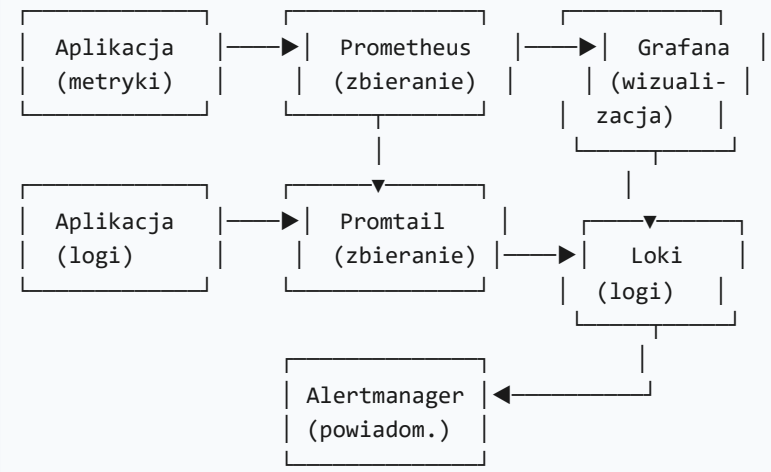
Po aktualizacji:

- [ ] Testy end-to-end przeszły (rejestracja, analiza, eksport)
- [ ] Monitoring pokazuje normalne metryki
- [ ] Użytkownicy poinformowani o zakończeniu
- [ ] Wersja udokumentowana w dzienniku operacyjnym

---

4. Monitoring

4.1. Architektura stosu monitoringu



Komponent	Rola	Port domyślny
Prometheus	zbieranie i przechowywanie metryk	9090
Grafana	wizualizacja, dashboardy	3000
Loki	agregacja logów	3100
Promtail	agent zbierający logi	—
Alertmanager	routing i eskalacja alertów	9093

4.2. Konfiguracja Prometheus

```yaml
# prometheus.yml
global:
  scrape_interval: 15s
  evaluation_interval: 15s

rule_files:
  - /etc/prometheus/rules/*.yml

alerting:
  alertmanagers:
    - static_configs:
        - targets: ["alertmanager:9093"]

scrape_configs:
  # FastAPI (backend)
  - job_name: testgen-backend
    metrics_path: /api/v1/metrics
    static_configs:
      - targets: ["backend:8000"]

  # Celery / Flower
  - job_name: testgen-flower
    static_configs:
      - targets: ["flower:5555"]

  # PostgreSQL (wymaga postgres_exporter)
  - job_name: testgen-postgres
    static_configs:
      - targets: ["postgres-exporter:9187"]

  # Redis (wymaga redis_exporter)
  - job_name: testgen-redis
    static_configs:
      - targets: ["redis-exporter:9121"]

  # RabbitMQ (wbudowany exporter prometheus)
  - job_name: testgen-rabbitmq
    static_configs:
      - targets: ["rabbitmq:15692"]

  # MinIO (wbudowane metryki)
  - job_name: testgen-minio
    metrics_path: /minio/v2/metrics/cluster
    static_configs:
      - targets: ["minio:9000"]

  # Node exporter (metryki systemowe)
  - job_name: node
    static_configs:
      - targets: ["node-exporter:9100"]
```

Retencja danych:

```
# Uruchomienie Prometheus z retencją 90 dni
prometheus \
  --config.file=/etc/prometheus/prometheus.yml \
  --storage.tsdb.retention.time=90d \
  --storage.tsdb.path=/prometheus
```

4.3. Konfiguracja Grafana

Źródła danych:

```
# grafana/provisioning/datasources/datasources.yml
apiVersion: 1
datasources:
  - name: Prometheus
    type: prometheus
    access: proxy
    url: http://prometheus:9090
    isDefault: true

  - name: Loki
    type: loki
    access: proxy
    url: http://loki:3100
```

Dashboardy (zalecane):

Dashboard	Zawartość
System	CPU, RAM, dysk, sieć per węzeł
Aplikacja	czas odpowiedzi, wskaźnik błędów, przepustowość (req/s)
Baza danych	aktywne połączenia, wolne zapytania, rozmiar bazy, replikacja
Celery / RabbitMQ	głębokość kolejek, czas przetwarzania zadań, błędy
MinIO	użycie dysku, ilość obiektów, przepustowość I/O

Przykładowy dashboard: `examples/grafana-dashboard.json`.

4.4. Konfiguracja Loki i Promtail

Promtail — zbieranie logów z kontenerów Docker:

```yaml
# promtail-config.yml
server:
  http_listen_port: 9080

positions:
  filename: /tmp/positions.yaml

clients:
  - url: http://loki:3100/loki/api/v1/push

scrape_configs:
  - job_name: docker
    docker_sd_configs:
      - host: unix:///var/run/docker.sock
        refresh_interval: 5s
    relabel_configs:
      - source_labels: ["__meta_docker_container_name"]
        target_label: container
      - source_labels: ["__meta_docker_container_log_stream"]
        target_label: stream
    pipeline_stages:
      - json:
          expressions:
            level: level
            msg: message
            timestamp: timestamp
      - labels:
          level:
      - timestamp:
          source: timestamp
          format: RFC3339
```

Przykłady zapytań LogQL:

```
# Błędy backendu w ostatniej godzinie
{container="testgen-backend-1"} |= "ERROR" | json | line_format "
{{.message}}"

# Wolne zapytania SQL (> 500 ms)
{container="testgen-backend-1"} |~ "slow query" | json | duration > 500ms

# Błędy Celery worker
{container=~"testgen-celery-worker.*"} |= "ERROR"

# Nieudane logowania
{container="testgen-backend-1"} |= "login_failed"
```

4.5. Reguły alertów

Plik: `examples/prometheus-rules.yml`.

```yaml
groups:
  - name: testgen-system
    rules:
      # Dysk > 85% (ostrzeżenie)
      - alert: DiskSpaceWarning
        expr: (1 - node_filesystem_avail_bytes /
         node_filesystem_size_bytes) > 0.85
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "Wykorzystanie dysku > 85% na {{ $labels.instance }}"
          description: "Partycja {{ $labels.mountpoint }}: {{ $value |
         humanizePercentage }}"

      # Dysk > 95% (krytyczny)
      - alert: DiskSpaceCritical
        expr: (1 - node_filesystem_avail_bytes /
         node_filesystem_size_bytes) > 0.95
        for: 2m
        labels:
          severity: critical
        annotations:
          summary: "KRYTYCZNE: dysk > 95% na {{ $labels.instance }}"

      # CPU > 80% (ostrzeżenie)
      - alert: HighCPUUsage
        expr: 100 - (avg by(instance)
         (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100) > 80
        for: 10m
        labels:
          severity: warning
        annotations:
          summary: "Wysokie zużycie CPU (> 80%) na {{ $labels.instance
         }}"

      # CPU > 95% (krytyczny)
      - alert: CriticalCPUUsage
        expr: 100 - (avg by(instance)
         (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100) > 95
        for: 5m
        labels:
          severity: critical

      # Pamięć > 85% (ostrzeżenie)
      - alert: HighMemoryUsage
        expr: (1 - node_memory_MemAvailable_bytes /
         node_memory_MemTotal_bytes) > 0.85
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "Wysokie zużycie pamięci (> 85%) na {{
         $labels.instance }}"

  - name: testgen-application
```

```yaml
rules:
  # Głębokość kolejki RabbitMQ > 1000
  - alert: QueueDepthWarning
    expr: rabbitmq_queue_messages > 1000
    for: 5m
    labels:
      severity: warning
    annotations:
      summary: "Kolejka {{ $labels.queue }} ma > 1000 wiadomości"

  # Wskaźnik błędów > 5%
  - alert: HighErrorRate
    expr: |
      sum(rate(http_requests_total{status=~"5.."}[5m]))
      / sum(rate(http_requests_total[5m])) > 0.05
    for: 5m
    labels:
      severity: warning
    annotations:
      summary: "Wskaźnik błędów HTTP 5xx > 5%"

  # Wskaźnik błędów > 10% (krytyczny)
  - alert: CriticalErrorRate
    expr: |
      sum(rate(http_requests_total{status=~"5.."}[5m]))
      / sum(rate(http_requests_total[5m])) > 0.10
    for: 2m
    labels:
      severity: critical

  # Backend niedostępny
  - alert: BackendDown
    expr: up{job="testgen-backend"} == 0
    for: 1m
    labels:
      severity: critical
    annotations:
      summary: "Backend TestGEN jest niedostępny"

  # PostgreSQL niedostępny
  - alert: PostgresDown
    expr: up{job="testgen-postgres"} == 0
    for: 1m
    labels:
      severity: critical

  # Celery — brak aktywnych workerów
  - alert: NoCeleryWorkers
    expr: flower_active_workers == 0
    for: 2m
    labels:
      severity: critical
    annotations:
      summary: "Brak aktywnych workerów Celery"
```

4.6. Alertmanager — routing i eskalacja

```yaml
# alertmanager.yml
global:
  resolve_timeout: 5m
  smtp_smarthost: "smtp.example.com:587"
  smtp_from: "alerts@testgen.example.com"
  smtp_auth_username: "alerts@testgen.example.com"
  smtp_auth_password: "<hasło>"
  smtp_require_tls: true

route:
  group_by: ["alertname", "severity"]
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 4h
  receiver: email-default

  routes:
    # Alerty krytyczne — natychmiast + Slack/PagerDuty
    - match:
        severity: critical
      receiver: critical-alerts
      repeat_interval: 1h

    # Alerty ostrzegawcze — email co 4h
    - match:
        severity: warning
      receiver: email-default
      repeat_interval: 4h

receivers:
  - name: email-default
    email_configs:
      - to: "devops@example.com"
        send_resolved: true

  - name: critical-alerts
    email_configs:
      - to: "devops-oncall@example.com"
        send_resolved: true
    # Opcjonalnie: Slack
    # slack_configs:
    #   - api_url: "https://hooks.slack.com/services/XXX/YYY/ZZZ"
    #     channel: "#testgen-alerts"
    #     title: '{{ .GroupLabels.alertname }}'
    #     text: '{{ .CommonAnnotations.summary }}'
```

5. Odtwarzanie po awarii (DR)

5.1. Strategia DR

Parametr	Cel	Źródło
RTO (maks. czas przywracania)	< 4 godziny	NFR31
RPO (maks. utrata danych)	< 1 godzina	NFR32
Częstotliwość testu DR	kwartalnie	najlepsza praktyka
Ośrodek zapasowy	opcjonalny (cold standby)	decyzja biznesowa

Priorytety przywracania (kolejność):

1. PostgreSQL (dane krytyczne)
2. RabbitMQ (kolejki zadań)
3. Redis (cache — stratny, odbudowuje się)
4. MinIO (pliki — wyniki analiz)
5. Aplikacja (backend, frontend, Celery)

5.2. Procedura odtwarzania

Cel: przywrócenie pełnej funkcjonalności systemu po katastrofalnej awarii.

Czas wykonania: < 4 godziny (RTO).

Krok	Czynność	Szacowany czas	Odpowiedzialny
1	Deklaracja incydentu	5 min	inżynier dyżurny
2	Ocena uszkodzeń	15 min	inżynier DevOps
3	Przywracanie infrastruktury	30–60 min	inżynier DevOps
4	Przywracanie bazy danych	30–60 min	DBA / DevOps
5	Przywracanie magazynu (MinIO)	15–30 min	inżynier DevOps
6	Weryfikacja systemu	15–30 min	QA / DevOps
7	Komunikacja z użytkownikami	5 min	PM / DevOps
Łącznie		**~2–3.5 h**	

Szczegółowe kroki:

```
# === KROK 1: Deklaracja incydentu ===
# Powiadomienie zespołu, utworzenie kanału incydentowego

# === KROK 2: Ocena uszkodzeń ===
# Sprawdź co działa, co nie:
docker compose ps              # lub: kubectl get pods -n testgen
docker compose logs --tail=100 # lub: kubectl logs -n testgen <pod>

# === KROK 3: Przywracanie infrastruktury ===
# A) Ten sam serwer (np. restart po awarii zasilania):
docker compose up -d

# B) Nowy serwer:
# Wykonaj kroki z 01-instalacja.md, sekcja 2.1–2.3
# Przywróć .env i klucze z kopii:
cp /backup/config/.env_XXXXXXXX .env
cp -r /backup/config/keys_XXXXXXXX/ backend/keys/

# === KROK 4: Przywracanie bazy danych ===
# Opcja A — pg_dump (RPO = czas od ostatniego backupu):
docker compose up -d postgres
sleep 15  # czekaj na healthcheck
docker compose exec -T postgres pg_restore \
  -U testgen -d testgen --clean --if-exists \
  < /backup/postgresql/daily/testgen_XXXXXXXX.dump

# Opcja B — PITR (RPO bliskie zeru):
# Patrz sekcja 1.2 — "Przywracanie PITR"

# === KROK 5: Przywracanie magazynu ===
docker compose up -d minio
sleep 15
docker compose exec minio mc mirror /backup/minio/testgen-results
         local/testgen-results
docker compose exec minio mc mirror /backup/minio/testgen-exports
         local/testgen-exports

# === KROK 6: Uruchomienie i weryfikacja ===
docker compose up -d

# Checklist weryfikacji:
curl -s http://localhost:8000/api/v1/health | python3 -m json.tool
curl -s -o /dev/null -w "%{http_code}" http://localhost:4200
docker compose exec redis redis-cli ping
docker compose exec rabbitmq rabbitmqctl status | head -5
docker compose exec minio mc admin info local

# === KROK 7: Komunikacja ===
# Powiadomienie użytkowników o przywróceniu usługi
```

5.3. Ośrodek zapasowy (DR site)

Cold standby (minimalny koszt):

- Osobna maszyna / klaster z zainstalowanym Docker / K8s
- Kopie zapasowe replikowane na magazyn zdalny (S3, NFS)
- W razie awarii: przywracanie z kopii (RTO: 2–4h)

Warm standby (szybsze przywracanie):

- PostgreSQL streaming replication na ośrodek zapasowy
- MinIO replikacja (`mc mirror --watch`)
- Przygotowane obrazy Docker w rejestrze
- W razie awarii: przełączenie DNS + promote repliki PG (RTO: 30–60 min)

Replikacja danych:

```
# PostgreSQL — ciągła replikacja strumieniowa
# Na ośrodku zapasowym (standby):
primary_conninfo = 'host=primary.example.com port=5432 user=replicator
         password=<hasło>'

# MinIO — ciągłe lustro
mc mirror --watch --overwrite local/testgen-results remote/testgen-
         results
```

5.4. Test DR — procedura kwartalna

Cel: potwierdzenie zdolności przywracania systemu w ramach RTO/RPO.

Harmonogram: co kwartał (styczeń, kwiecień, lipiec, październik).

Procedura:

1. **Przygotowanie** (dzień przed testem)
    - Powiadomienie zespołu
    - Przygotowanie środowiska testowego (oddzielna maszyna / przestrzeń nazw)
    - Weryfikacja dostępności kopii zapasowych
2. **Wykonanie testu**
    - Przywrócenie systemu z kopii na czystym środowisku
    - Zmierzenie czasu każdego kroku
    - Weryfikacja integralności danych
3. **Raport**
    - Czas przywracania vs. RTO
    - Utrata danych vs. RPO
    - Problemy napotkane i wnioski
    - Działania naprawcze

Szablon raportu:

```
# Raport testu DR — Q[N] [ROK]

**Data:** [data]
**Prowadzący:** [imię nazwisko]
**Środowisko:** [opis]

## Wyniki

| Metryka | Cel | Zmierzone | Status |
|---------|-----|-----------|--------|
| RTO | < 4h | [czas] | OK / PRZEKROCZONO |
| RPO | < 1h | [czas od ostatniego backup] | OK / PRZEKROCZONO |

## Przebieg

| Krok | Opis | Czas startu | Czas końca | Status |
|------|------|-------------|------------|--------|
| 1 | Przywracanie infra | | | |
| 2 | Przywracanie PG | | | |
| 3 | Przywracanie MinIO | | | |
| 4 | Uruchomienie aplikacji | | | |
| 5 | Weryfikacja | | | |

## Problemy

1. [opis problemu i rozwiązania]

## Wnioski i działania

1. [działanie naprawcze — osoba odpowiedzialna — termin]
```

5.5. Checklist DR

Gotowość (ciągła):

- ☐ Kopie zapasowe tworzone zgodnie z harmonogramem
- ☐ Kopie przenoszone na magazyn zdalny
- ☐ Ośrodek zapasowy (jeśli warm standby) — replikacja działa
- ☐ Dokumentacja DR aktualna
- ☐ Kontakty alarmowe aktualne
- ☐ Klucze i sekrety dostępne z kopii

Incydent DR:

- ☐ Incydent zadeklarowany, zespół powiadomiony
- ☐ Uszkodzenia ocenione
- ☐ Infrastruktura przywrócona
- ☐ Baza danych przywrócona i zweryfikowana
- ☐ Magazyn plików przywrócony
- ☐ Aplikacja uruchomiona, healthchecks przechodzą
- ☐ Testy krytyczne (logowanie, analiza, eksport) pozytywne
- ☐ Użytkownicy poinformowani o przywróceniu usługi

- [ ] Raport post-mortem przygotowany w ciągu 48h

Powiązane dokumenty

Dokument	Opis
`01-instalacja.md`	instalacja i konfiguracja (Story 9.6)
`getting-started.md`	szybki start dla programistów
`environment-setup.md`	pełna lista zmiennych środowiskowych
`examples/`	skrypty, konfiguracje, szablony